

Week_08_Quiz-jeg2253

November 15, 2025

1 Week 8 Quiz

1.1 Jayat Gonzalez Palomeras - jeg2253

1.1.1 Due Mon. Nov 18th, 11:59pm ET

In this quiz we'll practice some data transformations.

1.1.2 Instructions

Replace the Name and UNI in cell above and the notebook filename

Replace all '_____' below using the instructions provided.

When completed, 1. make sure you've replaced Name and UNI in the first cell and filename 2. Kernel -> Restart & Run All to run all cells in order 3. Print Preview -> Print (Landscape Layout) -> Save to pdf 4. post pdf to GradeScope

```
[1]: # import numpy as np and pandas as pd
import numpy as np
import pandas as pd
```

```
[2]: # Read in data from data/week8_housing_data.csv and store as dataframe df.
# This data includes a datetime column DocumentDate.
# Use parse_dates to parse this column into datetimes
# Print df.info() to see the number of rows, column names, column datatypes,
# and amount of missing data.
df = pd.read_csv('../data/week8_housing_data.csv', parse_dates=['DocumentDate'])
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   DocumentDate    500 non-null   datetime64[ns]
1   PropertyType    478 non-null   object
2   SqFtLot         489 non-null   float64
dtypes: datetime64[ns](1), float64(1), object(1)
memory usage: 11.8+ KB
```

```
[3]: # If we run df.duplicated() we get a vector of booleans that indicate
      ↪ duplicated rows.
      # Use df.duplicated() with default values and .sum() to assert that there are 3
      ↪ duplicated rows
      assert df.duplicated().sum() == 3
```

```
[4]: # Use drop_duplicates() to drop the duplicated rows.
      # Compare the entire row (subset=None) and keep the first duplicate
      ↪ (keep='first') (the defaults)
      # Store back into df
      df = df.drop_duplicates()

      # Confirm that the correct number of rows have been dropped
      assert df.shape[0] == 497
```

```
[5]: # Before continuing, note there is a missing SqFtLot in the first row
      df.head(3)
```

```
[5]:   DocumentDate  PropertyType  SqFtLot
0    2006-11-21  Single Family      NaN
1    2007-04-16    Townhouse    937.0
2    2006-01-18           NaN   13167.0
```

```
[7]: # From the .info() above, we see there are missing values in SqFtLot.
      # Before we fill this column, create a new dummy column 'SqFtLot_missing' in
      ↪ df.
      # This column should contain integers, 1 for missing, 0 for not missing.
      # Use .isna() and .astype(int) to create the 'SqFtLot_missing' column.
      df['SqFtLot_missing'] = df['SqFtLot'].isna().astype(int)

      # Assert that the number of 1's in the SqFtLot_missing column equals the number
      ↪ of missing values in SqFtLot
      assert df['SqFtLot_missing'].sum() == df['SqFtLot'].isna().sum()

      # Assert that the dtype of SqFtLot_missing is int
      assert df.SqFtLot_missing.dtype == int
```

```
[8]: # Now fill the missing values in df.SqFtLot with the mean of the SqFtLot column.
      # Use .fillna() and .mean()
      # Store back into the existing SqFtLot column.
      df['SqFtLot'] = df['SqFtLot'].fillna(df['SqFtLot'].mean())

      # Assert that the SqFtLot column no longer contains any missing values (number
      ↪ of missing values == 0)
      assert df.SqFtLot.isna().sum() == 0
```

```
[9]: # The missing SqFtLot should now be filled
df.head(3)
```

```
[9]: DocumentDate  PropertyType  SqFtLot  SqFtLot_missing
0    2006-11-21  Single Family  13801.04321             1
1    2007-04-16    Townhouse    937.00000             0
2    2006-01-18           NaN  13167.00000             0
```

```
[10]: # There are also missing values in PropertyType.
# Since 'PropertyType' is categorical, let's treat MISSING as another
# category.
# Fill the empty values in PropertyType with the string 'MISSING'.
# Store back into the existing PropertyType column.
df['PropertyType'] = df['PropertyType'].fillna('MISSING')

# Call .value_counts() on the PropertyType column
# to see how many of each category exist in the dataframe.
# We should see 22 MISSING values
df.PropertyType.value_counts()
```

```
[10]: PropertyType
Single Family    455
MISSING           22
Townhouse        12
Multiplex         8
Name: count, dtype: int64
```

```
[11]: # Confirm we have no missing data by asserting that the sum of df.isna() over
# rows and columns is equal to 0.
assert df.isna().sum().sum() == 0

# Print df.info() to visually confirm there are no missing values as well
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 497 entries, 0 to 499
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   DocumentDate    497 non-null   datetime64[ns]
1   PropertyType     497 non-null   object
2   SqFtLot          497 non-null   float64
3   SqFtLot_missing  497 non-null   int64
dtypes: datetime64[ns](1), float64(1), int64(1), object(1)
memory usage: 19.4+ KB
```

```
[12]: # Before we one hot encode PropertyType, print the first 3 rows of df and note
# their PropertyType values
```

```
df.head(3)
```

```
[12]: DocumentDate  PropertyType  SqFtLot  SqFtLot_missing
0    2006-11-21  Single Family  13801.04321          1
1    2007-04-16    Townhouse    937.00000          0
2    2006-01-18      MISSING  13167.00000          0
```

```
[13]: # Transform the categorical feature PropertyType using pd.get_dummies().
# Note that we can call get_dummies on the entire dataframe and only
# categorical features will be transformed.
# Store the result of get_dummies into df_new
df_new = pd.get_dummies(df)

# Print out the first 3 rows of df_new to see the result.
df_new.head(3)
```

```
[13]: DocumentDate  SqFtLot  SqFtLot_missing  PropertyType_MISSING \
0    2006-11-21  13801.04321          1          False
1    2007-04-16    937.00000          0          False
2    2006-01-18  13167.00000          0          True

PropertyType_Multiplex  PropertyType_Single Family  PropertyType_Townhouse
0          False          True          False
1          False          False          True
2          False          False          False
```

```
[ ]:
```